

헬로우 투 헤일로

이종민[○], 이주형, 장하영

한국공학대학교 게임공학과

{powerspirit127, jh10590326, hayoungtuk}@gmail.com

요 약

본 프로젝트는 Rust 언어^[1]의 게임 개발 실용성을 평가하고자 “Blue Archive”^[9] IP 기반의 TPS 게임을 개발했다. Rust의 Tokio(네트워킹)^[2], wgpu(그래픽스)^[3]를 핵심 기술로 사용했다. 평가 결과, Rust는 소유권 시스템을 통한 높은 메모리 안전성 확보와 Cargo를 통한 효율적인 패키지 관리 측면에서 장점을 보였다. 반면, 높은 학습 곡선과 기존 C++ 디자인 패턴 적용의 어려움, unsafe 및 순환 참조의 잠재적 위험은 한계로 식별되었다. 게임 자체는 서버 권위적 모델로 인한 네트워크 지연과 콘텐츠 완성도 부족이 한계점으로 남았으며, 향후 서버 모델 개선을 과제로 제시한다.

1. 서 론

본 프로젝트의 가장 주된 제작 목적은 기존 게임 개발의 주류 언어인 C++를 대신하여, 안정성과 성능 면에서 최근 주목받는 Rust 언어^[1]를 게임 개발에 실제 적용해 보는 데 있다. 이를 통해 게임 개발 측면에서 Rust 언어가 가지는 실용성과 기술적 한계를 구체적으로 평가하고자 하였다. 이러한 목표를 달성하기 위해 Rust 언어를 기반으로 비동기 런타임을 위한 Tokio^[2]크레이트(Crate)와 그래픽스 API를 위한 wgpu^[3]크레이트를 사용하여 멀티플레이어 게임을 개발하였다.

본 작품은 ‘TPS(3인칭 슈터) 팀플레이 히어로 슈팅’ 장르의 게임이다. 최대 10명의 플레이어가 두 팀으로 나뉘어, 각기 다른 고유 스킬과 무기를 가진 “Blue Archive”의 캐릭터를 직접 조작한다. 게임의 승리 조건은 ‘거점 점령’ 방식으로, 맵의 특정 거점을 점령하여 포인트를 획득하는 것을 목표로 한다. 목

표 점수에 먼저 도달하거나, 제한 시간이 초과되었을 때 더 많은 점수를 획득한 팀이 최종적으로 승리하게 된다.

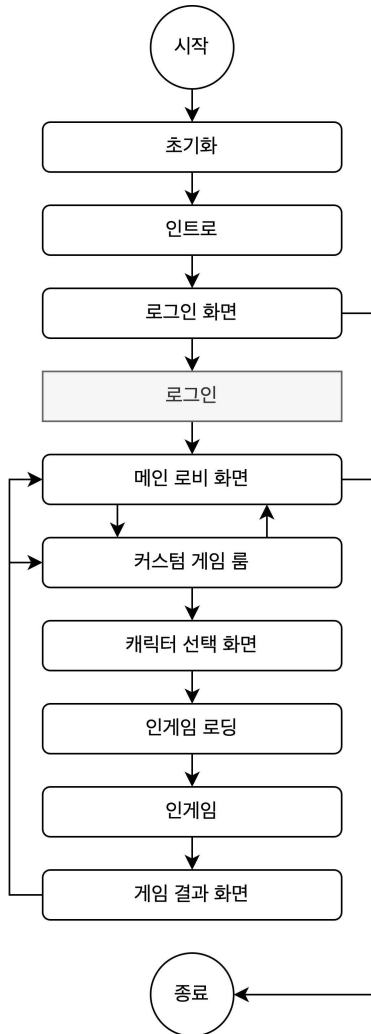
본 게임의 구동 플랫폼은 PC 환경이며, Windows 10(20H2 이상) 및 macOS 11(Big Sur 이상)을 지원한다.

본 보고서는 다음과 같이 구성된다. 2장에서는 본 게임의 구체적인 개발 내용과 시스템 흐름, 개발 방법론을 서술한다. 3장에서는 개발 결과물에 대한 요약과 한계점 및 향후 과제를 제시하며, 마지막 4장 부록에서는 게임의 설치 및 실행 환경, 상세한 진행 방법 등을 기술한다.

2. 개발 내용

본 2장에서는 프로젝트에 사용된 개발 환경 및 방법론, 그리고 Rust를 기반으로 구현한 핵심 기술들과 주요 게임 시스템에 대해 상세히 서술한다.

2.1. 게임 전체 흐름도



[그림 1] 게임 전체 흐름도

2.2. 개발 환경 및 아키텍처

본 프로젝트는 Windows와 macOS 환경에서 VSCode와 Git을 기반으로 개발하였다. 원작 “Blue Archive”의 리소스 활용을 위해 Unity Editor를 보조 도구로 사용하였다. Unity 상에서 맵 오브젝트 배치 등을 구성하고 해당 데이터를 JSON 포맷으로 추출하여,

Rust 프로젝트의 에셋으로 통합하는 파이프라인을 구축하였다.

개발 언어는 Rust^[1]이며, 핵심 라이브러리(크레이트)는 다음과 같다.

(1) wgpu^[3]: Rust의 안정성 기반 크로스플랫폼 고성능 그래픽스 렌더링을 위해 선택하였으며 클라이언트 구현에 사용하였다.

(2) Tokio^[2]: 검증된 비동기 네트워크 런타임으로, 멀티플레이어 서버 구현에 사용하였다.

2.3. 개발 방법론

본 프로젝트는 3인 팀으로 구성되었으며, 클라이언트 개발 및 프로젝트 총괄(이종민)과 서버 개발(이주형, 장하영)로 역할을 명확히 나누어 각자의 파트를 개발하였다.

프로젝트 관리는 GitHub를 중심으로 이루어졌다. 팀장이 GitHub Issues를 통해 작업을 분배하고 기한을 설정하여 진행 상황을 추적하였다. 버그 리포트 및 피드백 역시 GitHub Issues를 통해 관리하였으며, 카카오톡 메시지를 통한 실시간 소통을 보조적으로 활용하였다.

2.4. 핵심 기술 구현

본 프로젝트는 C++ 대신 Rust 언어의 실용성 검증에 주요 목표로 하므로, Rust를 기반으로 한 핵심 시스템 구현 내용을 상세히 기술한다.

2.4.1. 비동기 네트워킹 구현

멀티플레이어 환경을 위해 Tokio를 사용한 비동기 네트워킹을 구현하였다. 본 게임은 서버 권위적(Server-Authoritative) 모델^[4]을

채택하였다. 이는 모든 클라이언트의 입력을 서버로 전송하고, 서버가 모든 연산(이동, 전투)을 처리하여 그 결과를 다시 클라이언트에 전파하는 방식이다.

초기 개발 단계에서 서버-클라이언트 모델 설계가 미흡하여 해당 방식을 채택하였으나, 다음과 같은 문제점이 식별되었다.

【문제점】

- (1) 캐릭터 움직임 지연(Lag): 서버의 데이터 갱신 주기(Tick rate)가 클라이언트에 직접적인 영향을 주었다.
- (2) 데이터 오버헤드: 모든 입출력과 상태를 서버가 처리하고 전파해야 하므로, 송수신되는 패킷의 양이 불필요하게 커지는 오버헤드가 발생하였다.

【해결 방안】

- (1) 데이터 압축: 캐릭터의 좌표(float)와 같은 연속적인 데이터를 0에서 65535 사이의 정수 값으로 정규화(Normalization)하여 데이터 크기를 줄였다.^[8]
- (2) 비트 플래그(Bit Flag): 여러 개의 상태 값을 하나의 정수형 데이터에 비트 플래그로 통합하여 패킷 효율을 높였다.

2.4.2. 그래픽스 구현

wgpu를 기반으로 구현한 주요 그래픽스 렌더링 기술을 다음과 같다.

- (1) 애니메이션 믹싱: ‘걷기’ 애니메이션과 ‘공격’ 애니메이션을 혼합하여, 플레이어가 이동 중에도 자연스럽게 공격할 수 있는 ‘이동 공격’ 동작을 생성하였다.

- (2) 절차적 애니메이션: 별도의 애니메이션 데이터 파일 없이, 코드 기반 계산을 통해 동적인 움직임을 구현하였다. (예: 점프 시 다리 움직임, 충구 방향에 따른 허리 각도 실시간 변경). 이는 특정 뼈(Bone) 노드에 실시간으로 계산된 회전 행렬을 적용하여 구현하였다.
- (3) 그림자 구현(Shadow Mapping): 플레이어 주변의 정적 및 동적 광원에 대한 Shadow Map을 텍스처에 렌더링한 후, 본 렌더링 패스에서 이 Shadow Map을 샘플링하여 실시간 그림자를 구현하였다.
- (4) 렌더링 최적화(정적 그림자 활용): 맵에 배치된 건물과 같이 움직이지 않는 고정된 오브젝트의 경우, 미리 그림자를 계산하여 텍스처에 구워내는 정적 그림자(Static Shadow)를 활용하였다. 플레이어와 일정 거리 이상 떨어져 동적 그림자의 영향이 적은 오브젝트는 이 정적 그림자를 사용하도록 처리했다. 이 방식을 통해, 씬(Scene)의 전체 드로우 콜(Draw Call) 횟수를 기존 1,810회에서 487회로 약 73% 감소시키는 의미 있는 성능 향상을 달성하였다.



[그림 2] 정적 그림자 텍스처

(5) 가중치 기반 정렬 없는 알파 블렌딩^[5]: 반투명 오브젝트(예: 이펙트)가 겹칠 때 발생하는 렌더링 순서 문제를 해결하기 위해, 멀티렌더 타겟(MRT) 기법을 활용한 정렬 없는 알파 블렌딩(OIT) 방식을 사용하여 시각적 품질을 높였다.

2.5. 주요 게임 시스템 구현

앞서 설명한 핵심 기술들을 바탕으로 구현된 주요 게임 시스템은 다음과 같다.

2.5.1. 캐릭터 시스템



[그림 3] 캐릭터 목록

플레이어는 “Blue Archive”의 캐릭터 중 하나를 선택하여 조작한다. 각 캐릭터는 원작(“Blue Archive”)의 고증을 따라 AR(돌격소총), SR(저격소총), SMG(기관단총) 등 각기 다른 고유 무기와 고유 스킬을 보유한다.

이러한 캐릭터별 기능적 차이는 Rust 코드 내에서 match 패턴을 주로 사용하여 구현하였다. (예: 캐릭터 ID나 무기 타입에 따라 match 문으로 분기하여 각기 다른 발사 로직, 스킬 로직, 능력치를 적용). 이를 통해 데이터와 로직을 명확하게 분리하여 관리할 수 있었다.

2.5.2. 전투 및 스킬 시스템

TPS 슈팅 장르의 핵심인 전투 및 스킬 로

직은 서버 권위적 모델을 기반으로 다음과 같이 구현하였다.

【충돌 판정(Collision Detection)】

투사체와 오브젝트간 충돌 판정은 AABB, OBB, 그리고 캐릭터를 위한 Y축 캡슐 충돌체를 복합적으로 사용하여 판정하였다.

【공격(Attack) 로직】

- (1) (클라이언트) 마우스 클릭(공격) 이벤트 발생 시, 서버로 공격 요청 패킷을 전송한다.
- (2) (서버) 패킷을 수신한 후, 게임 월드 갱신 시점에 해당 플레이어 위치에 총알(투사체) 오브젝트를 생성한다.
- (3) (서버) 매 틱(Tick)마다 총알이 다른 플레이어, 건물 등과 충돌하는지를 확인하고, 충돌 시 피격 판정을 처리한다.

【스킬(Skill) 로직】

- (1) (클라이언트) 스킬 키 입력 이벤트 발생 시, 서버로 스킬 사용 요청 패킷을 전송한다.
- (2) (서버) 패킷을 수신한 후, 해당 플레이어 상태를 ‘스킬 시전 중’으로 변경한다.
- (3) (서버) 플레이어의 상태 변경이 확인되면, 서버 로직에 따라 스킬 효과를 발동시키고 이를 모든 클라이언트에 전파한다.

2.5.3. 게임 모드 구현

본 게임의 핵심 승리 조건인 ‘거점 점령’은 [2.4.1]에서 설명한 서버 권위적 모델을 기반으로 구현되었다.

【데이터 관리】 각 거점의 상태는 서버 측에서 CapturePoint라는 별도의 구조체를 통해 관리된다. 이 구조체는 현재 거점을 점령 중인 팀과 팀별로 누적된 점령 게이지 정보를 포함한다.

【핵심 로직】서버는 매 게임 월드 갱신(Tick) 시점마다 각 거점의 트리거 영역 내 플레이어 수를 확인하여 다음 로직을 수행한다.

- (1) 단독 점령: 특정 거점 내에서 A팀 플레이어만 존재할 경우, A팀의 ‘점령도’ 게이지를 증가시킨다.
- (2) 점수 획득: A팀 ‘점령도’가 100%에 도달하여 점령에 성공하면, A팀의 팀 전체 ‘점령 점수’를 증가시킨다.
- (3) 경합 상태: 거점 내에 A팀과 B팀 플레이어가 동시에 존재하는 경우, ‘중립’ 상태로 판정하여 점령도와 점수 획득이 모두 정지되고 현재 상태가 유지된다.

【세부 수치】게임의 속도감과 플레이 타임을 조절하기 위해 점령 관련 수치는 다음과 같이 설정하였다.

- (1) 점령도(Capture Progress): 밀리 초(ms) 당 1점씩 획득하며, 최대 15,000점(완료까지 15초 소요)으로 설정하였다.
- (2) 점령 점수(Game Score): 거점 점령 성공시, 밀리초 당 1점의 점수를 획득하며, 한 팀이 50,000점을 먼저 획득하면 승리한다.

【UI 동기화】서버는 갱신된 점수와 각 거점의 상태(CapturePoint 데이터)를 모든 클라이언트에게 주기적으로 전송한다. 클라이언트는 이 데이터를 수신하여 플레이어의 HUD(헤드업 디스플레이)에 표시되는 점수판과 거점 상태 아이콘을 실시간으로 갱신한다.

3. 결 론

본 장에서는 “Blue Archive”^[9] IP를 활용한 Rust 기반 TPS 게임 개발 프로젝트의 결과를 요약하고, 핵심 목표였던 Rust 언어의 실용성 평가를 서술한다. 또한, 프로젝트의 한계점을 명확히 밝히고 향후 발전 방향을 제시한다.

3.1. 프로젝트 요약



[그림 4] 게임 스크린샷

본 프로젝트는 Rust 언어^[1]의 게임 개발 실용성 검증을 목표로, “Blue Archive” IP를 활용한 3인칭 팀플레이 슈터(TPS) 게임을 개발하였다. Rust의 Tokio^[2]와 wgpu^[3]크레이트를 기반으로 멀티플레이어 네트워킹과 크로스플랫폼 렌더링을 구현하였으며, ‘거점 점령’ 게임 모드를 완성하였다. 개발 과정은 GitHub Issues를 통해 체계적으로 관리되었으며, 렌더링 최적화(드로우 콜 73% 감소) 및 패킷 최적화 등의 기술적 성과를 달성하였다.

3.2. Rust 언어의 실용성

평가 및 성과

프로젝트의 핵심 목표였던 ‘Rust 언어의 게임 개발 실용성’은 다음과 같이 평가된다.

【주요 성과(장점)】

- (1) 높은 메모리 안전성: 컴파일 시점에 소유권 규칙을 엄격하게 검사하므로, 널 포인터 역참조나 데이터 경쟁(Data Race)과 같은 고질적인 메모리 관련 버그를 사전에 효과적으로 방지할 수 있다.^[6]
- (2) 명시적인 오류 처리: Result<T, E> 열거형은, 함수가 반환할 수 있는 모든 오류 케이스를 컴파일 시점에 명시적으로 처리하도록 강제한다. 이는 C/C++와 같은 언어에서 null 반환 값이나 특정 오류 코드의 처리를 개발자가 누락함으로써 발생할 수 있는 ‘처리되지 않은 오류(Unhandled Error)’ 관련 버그를 원천적으로 방지하는 효과를 가진다. 결과적으로, 이는 잠재적인 런타임 결함을 감소시킨다. 최근 연구에 따르면 Rust는 메모리 안전성을 보장하면서도 C++ 대비 대등한 연산 성능을 유지함^[7]이 입증되었으며, 본 프로젝트에서도 이러한 이점을 확인할 수 있었다.
- (3) 예측 가능한 리소스 해제: 소유권이 이전되거나 스코프(Scope)를 벗어날 때 리소스가 해제됨이 보장되므로, 메모리 해제 타이밍을 예측하기 쉽다.

【기술적 한계(고려 사항)】

- (1) 기존 패턴 적용의 어려움: 소유권과 빌림(Borrowing) 규칙 때문에, 기존 C++ 등에서 사용하던 객체 지향 디자인 패턴(예: 상속 기반 다형성)이나 이중 연결 리스트 등을 직접 적용하기 어려운 경우가 많았다.
- (2) 잠재적 메모리 문제: Rust의 메모리 안정성은 컴파일 타임에 보장되지만, unsafe 키워드를 사용한 코드 블록이나, 순환 참조(Circular Reference)가 발생하는 경우 런타

임에서 메모리 누수가 발생할 수 있음을 확인하였다.

3.3. 한계점 및 향후 과제

본 프로젝트는 다음과 같은 한계점을 지니며, 이를 개선하기 위한 향후 과제를 제시한다.

【한계점】

- (1) 네트워크 지연 문제: [2.4.1]에서 기술한 서버 권위적(Server-Authoritative) 모델^[4]을 채택함에 따라, 패킷 최적화를 적용했음에도 불구하고 사용자가 체감하는 네트워크 지연(Lag)문제를 완벽히 해결하지 못하였다.
- (2) 콘텐츠 완성도 부족: 개발 일정의 한계로 당초 기획했던 “Blue Archive” 캐릭터를 구현하지 못하였다. 또한, 현재 구현된 캐릭터들의 스킬 이펙트(VFX)의 완성도가 전반적으로 부족하다.

【향후 과제】

- (1) 서버 모델 개선: 네트워크 지연 문제를 근본적으로 해결하기 위해, 현재 서버 모델을 클라이언트 측 예측(Client-Side Prediction) 및 보간(Interpolation), 서버 되감기(Server Rewind)와 같은 고급 기법을 도입한 모델로 수정하여 게임의 완성도를 높이려 한다.
- (2) 콘텐츠 확장: 구현하지 못한 추가 캐릭터를 구현하고, 기존 이펙트의 품질을 높여 게임의 시각적 완성도를 보강할 계획이다.
- (3) 신규 게임 모드 개발: ‘거점 점령’ 모드 외에 ‘팀 데스매치’나 PVE 협동 모드와 같은 새로운 게임 모드를 제작하여 콘텐츠를 다각화 한다.

4. 부 록

본 부록은 개발된 게임의 실제 설치 및 실행에 필요한 기술적인 정보와 사용자 매뉴얼을 제공하는 것을 목적으로 한다.

4.1. 게임 실행 환경

항목	내용
운영체제	Windows, macOS
저장공간	최소 1GB
RAM	최소 4GB
CPU	SSE4.1 지원 x86_64 Neon 지원 Apple Silicon
GPU	BC 텍스처 압축 포맷 지원 하드웨어

[표 1] 게임 사양

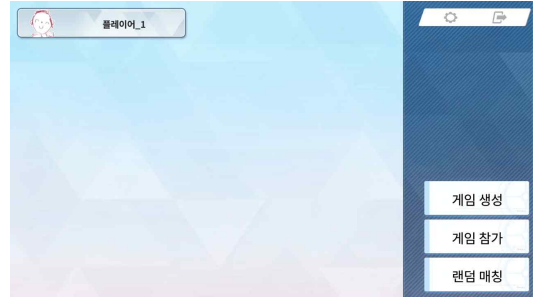
4.2. 게임 설치

본 게임의 설치 과정은 별도의 인스톨러(Installer) 없이 압축 파일을 해제하는 방식으로 진행된다. 설치 단계는 다음과 같다.

- (1) 압축된 zip 파일을 다운로드 한다.
- (2) 다운로드한 zip 파일의 압축을 해제한다.
- (3) 압축 해제된 폴더 내의 README.txt 파일에 명시된 설명에 따라 게임 실행 파일을 실행한다.

4.3. 게임 진행 방법

본 장에서는 [4.2. 게임설치]를 성공적으로 완료한 사용자가 게임을 실행하여 원활하게 플레이하는 데 필요한 모든 정보를 제공한다.



[그림 5] 게임 로비 장면

사용자가 로그인 화면을 통과하면 [그림 5]와 같이 메인 로비(Main Lobby)로 진입한다. 로비 화면은 사용자가 세션에 참여하기 위한 인터페이스를 제공하며, 기능은 다음과 같다.

- (1) 게임 생성: 사용자가 직접 새로운 게임 세션(방)을 생성하여 호스트(Host) 역할을 수행하는 기능이다.
- (2) 게임 참가: 사용자가 참가를 원하는 특정 세션을 입력하여 입장하는 기능이다.
- (3) 랜덤 매칭: 별도의 세션 선택 과정 없이, 시스템이 자동으로 입장 가능한 세션을 탐색하여 사용자를 배치하는 기능이다.



[그림 6] 게임 대기실 화면

세션에 참가한 사용자는 게임이 시작되기 전 [그림 6]과 같이 ‘게임 대기실’ 화면으로

진입한다. 이 화면은 게임을 시작하기 위한 준비 단계이며, 인터페이스의 주요 구성요소는 다음과 같다.

- (1) 참가자 목록: 현재 대기실에 접속한 모든 사용자의 목록을 실시간으로 표시한다.
- (2) 준비 버튼: 일반 참가자는 준비 버튼을 클릭하여, 게임을 시작할 준비가 완료되었음을 다른 사용자에게 알릴 수 있다.
- (3) 옵션 토글: 세션을 생성한 호스트에게는 게임 설정을 변경할 수 있는 제어 권한이 부여된다. 호스트는 제공된 ‘옵션 토글’을 조작하여 게임 변수를 설정할 수 있다.



[그림 7] 캐릭터 선택 화면

대기실에서 호스트가 매치를 시작하면, 모든 참가자는 [그림 7]과 같이 ‘캐릭터 선택 화면’으로 전환된다. 이 화면은 플레이어가 해당 게임 세션에서 직접 조작할 캐릭터를 선택하는 핵심 단계이며 구성요소는 다음과 같다.

- (1) 캐릭터 선택: 사용자는 화면에 제시된 캐릭터 목록 중 하나를 선택한다.
- (2) 게임 진입: 대기실 내 모든 플레이어가 캐릭터 선택을 완료한 것이 서버에 의해 확인되면, 모든 클라이언트를 인게임 장면으로 로드하여 실제 게임 플레이를 시작한다.



[그림 8] 인게임 화면

캐릭터 선택을 완료하고 [그림 8]과 같이 인게임(In-Game) 장면에 진입하면, 플레이어는 원활한 게임 진행을 위한 다음과 같은 정보를 확인할 수 있다.

- (1) 상단: 화면 상단 중앙에는 게임의 주요 목표인 ‘거점 점령 게이지’의 현재 상태와 ‘남은 시간’이 표시된다.
- (2) 좌측 하단: 플레이어 본인 캐릭터의 ‘체력 (HP)’ 상태가 표시된다.
- (3) 우측 하단: 무기의 ‘잔여 탄약 수’와 사용 가능한 ‘스킬 게이지’의 충전 상태가 표시된다.
- (4) 좌측: 플레이어 자신을 제외한 소속 팀원들의 ‘체력’ 및 ‘스킬 게이지’ 현황이 실시간으로 표시된다.

4.4. 제약 사항 및 문제 해결

【제약 사항】

- (1) 동일한 식별자로 로그인 할 경우 클라이언트에서 크래시가 발생한다.

【문제 해결을 위한 연락처】

팀장 이종민: powerspirit127@gmail.com

5. 참고 문헌

- [1] S. Klabnik and C. Nichols, *The Rust Programming Language*, San Francisco, CA: No Starch Press, 2018.
- [2] Tokio Contributors, "Tokio: An asynchronous Rust runtime," [Online]. Available: <https://tokio.rs/>. [Accessed: Dec. 16, 2025].
- [3] wgpu Developers, "wgpu: Portable graphics library for Rust," [Online]. Available: <https://wgpu.rs/>. [Accessed: Dec. 16, 2025].
- [4] G. Fiedler, "What Every Programmer Needs to Know About Game Networking," Gaffer On Games, 2010. [Online]. Available: https://gafferongames.com/post/what_every_programmer_needs_to_know_about_game_networking/. [Accessed: Dec. 16, 2025].
- [5] M. McGuire and L. Bavoil, "Weighted Blended Order-Independent Transparency," *Journal of Computer Graphics Techniques (JCGT)*, vol. 2, no. 2, pp. 122 - 141, 2013.
- [6] Microsoft Security Response Center, "A proactive approach to more secure code," [Online]. Available: <https://msrc.microsoft.com/blog/2019/07/a-proactive-approach-to-more-secure-code/>. [Accessed: Dec. 16, 2025].
- [7] M. Costanzo, E. Rucci, M. Naiouf, and A. De Giusti, "Performance vs Programming Effort between Rust and C on Multicore Architectures: Case Study in N-Body," in *Proc. XLVII Latin American Computing Conference (CLEI)*, 2021.
- [8] G. Fiedler, "Snapshot Compression," Gaffer On Games, [Online]. Available: https://gafferongames.com/post/snapshot_compression/. [Accessed: Dec. 16, 2025].
- [9] Nexon Games, *Blue Archive* [Video Game]. Seoul, Korea: Nexon, 2021.